

## **CLOUD INFRASTRUCTURE AUTOMATION USING TERRAFORM & AWS**

**Services:** EC2, RDS, VPC, Terraform

**Description:** Automate the provisioning of an application stack with Terraform. The stack includes EC2 instances in a VPC with a public and private subnet, a load balancer, and an RDS database.

**Skills:** Infrastructure as Code (IaC), Terraform, VPC setup

### **1. Definition**

Cloud Infrastructure Automation using Terraform is a project that demonstrates how infrastructure resources in **Amazon Web Services** can be provisioned automatically using **Terraform**.

Instead of manually creating cloud resources through the AWS console, Terraform allows developers and DevOps engineers to define infrastructure as code (IaC). These configurations automatically create and manage cloud components such as VPCs, EC2 instances, load balancers, and databases.

In this project, Terraform is used to deploy a scalable web application architecture that includes a Virtual Private Cloud (VPC), public subnets, an Application Load Balancer, an EC2 web server, and a database layer.

### **2. Scope of the Project**

The scope of this project includes automating the provisioning of cloud infrastructure required for hosting a web application.

The key objectives are:

- Automate AWS resource creation using Terraform
- Implement Infrastructure as Code (IaC)
- Deploy a scalable and highly available architecture
- Configure networking components like VPC, subnets, and route tables
- Deploy an EC2 instance to host a web server
- Configure an Application Load Balancer to distribute traffic
- Connect the load balancer with the EC2 instance using a target group

This project demonstrates how DevOps engineers can manage infrastructure efficiently using automation tools.

### 3. Methodology

The methodology used in this project follows the **Infrastructure as Code (IaC) approach**.

The implementation process involves the following steps:

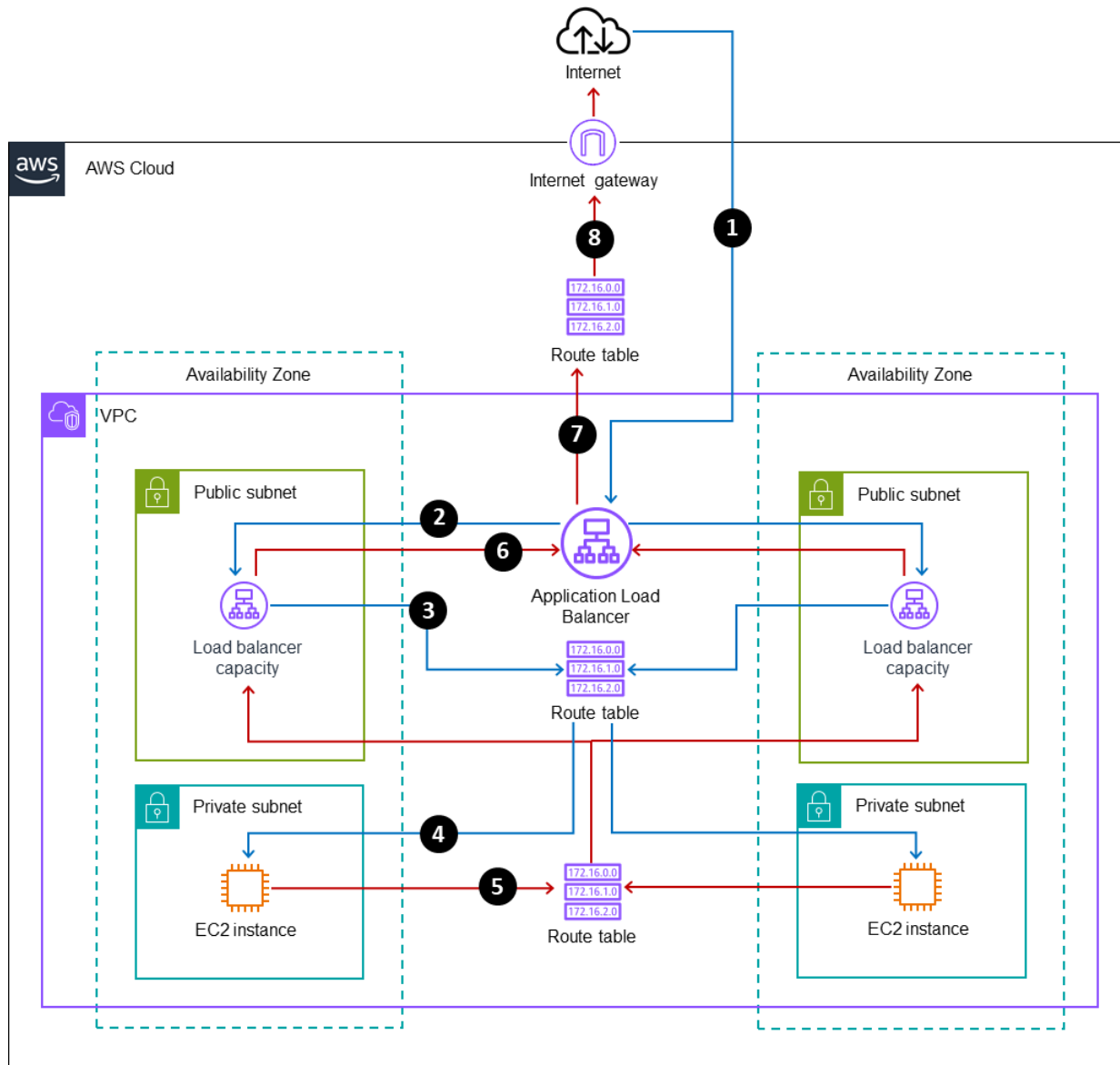
1. Installing Terraform and configuring AWS CLI.
2. Writing Terraform configuration files to define infrastructure.
3. Initializing Terraform to download provider plugins.
4. Planning the infrastructure changes using Terraform.
5. Applying the Terraform configuration to create AWS resources.
6. Deploying and testing the application through the load balancer.
7. Troubleshooting and resolving deployment issues.

This approach ensures repeatability, consistency, and scalability of infrastructure deployment.

### 4. Architecture Diagram / Flow

Components:

- VPC
- Public Subnets
- Internet Gateway
- Route Tables
- Security Groups
- EC2 Instance
- Application Load Balancer
- Target Group
- Terraform Automation



### Architecture Flow:

```

User
↓
Application Load Balancer
↓
Target Group
↓
EC2 Web Server (Apache)
↓
VPC Network
  
```

## **5. Explanation of Services Used**

### **Virtual Private Cloud (VPC)**

A VPC provides an isolated virtual network within AWS where cloud resources are deployed. It allows control over IP address ranges, subnets, routing, and security.

Purpose:

To create a secure and customizable network environment for hosting infrastructure resources.

### **Subnets**

Subnets divide a VPC into smaller networks. In this project, two public subnets are created across different availability zones.

Purpose:

To improve availability and allow the load balancer to operate across multiple zones.

### **Internet Gateway**

An Internet Gateway enables communication between instances in the VPC and the internet.

Purpose:

Allows external users to access the web application through the load balancer.

### **Security Groups**

Security groups act as virtual firewalls controlling inbound and outbound traffic.

Purpose:

To allow necessary traffic such as:

- HTTP (port 80)
- SSH (port 22)

### **EC2 Instance**

An EC2 instance is a virtual server in AWS used to run applications.

Purpose:

To host the Apache web server and serve the web application.

### **Application Load Balancer (ALB)**

The Application Load Balancer distributes incoming web traffic across multiple targets such as EC2 instances.

Purpose:

To ensure high availability and efficient traffic distribution.

### **Target Group**

A target group routes requests from the load balancer to registered EC2 instances.

Purpose:

To manage and monitor the health of backend instances.

### **Terraform**

Terraform is an Infrastructure as Code tool used to provision and manage cloud infrastructure automatically.

Purpose:

To automate the creation of AWS resources using configuration files.

## **6. Step-by-Step Procedure**

### **Step 1: Install Terraform**

Terraform was installed on the system and verified using:

`terraform -version`

This confirms that Terraform is ready to use.

```
ubuntu@ip-172-31-6-252:~/terraform-aws-project$ terraform --version
Terraform v1.14.6
on linux_amd64
+ provider registry.terraform.io/hashicorp/aws v6.35.1
ubuntu@ip-172-31-6-252:~/terraform-aws-project$
```

### **Step 2: Configure AWS CLI**

AWS credentials were configured using:

`aws configure`

This allows Terraform to authenticate with AWS.

### Step 3: Create Terraform Project Structure

A project folder was created containing Terraform configuration files such as:

```
provider.tf
vpc.tf
ec2.tf
alb.tf
security.tf
```

Each file defines specific infrastructure components.

```
ubuntu@ip-172-31-6-252:~/terraform-aws-project$ ls
alb.tf  ec2.tf  igw.tf  provider.tf  rds.tf  rtb.tf  security.tf  terraform.tfstate  terraform.tfstate.backup
vpc.tf
ubuntu@ip-172-31-6-252:~/terraform-aws-project$
```

### Step 4: Define AWS Provider

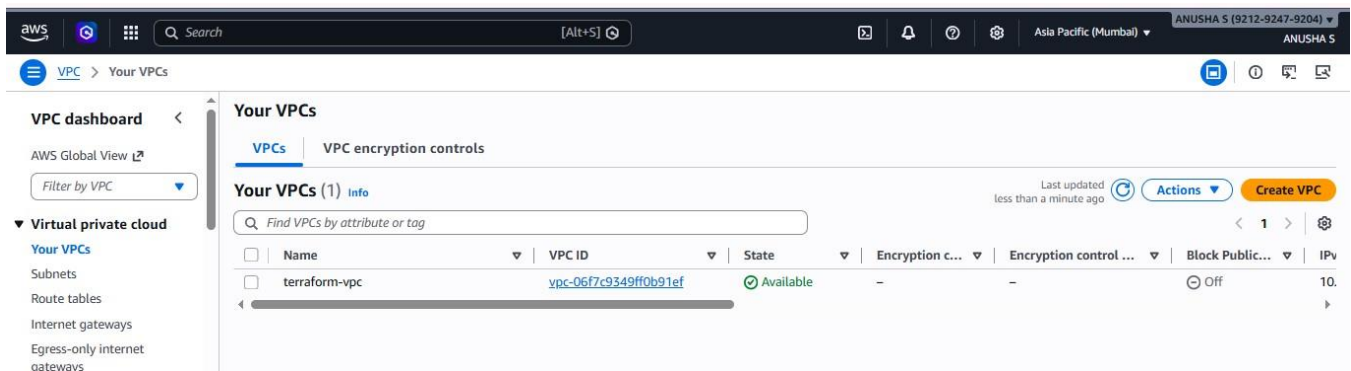
The AWS provider was configured in Terraform to specify the AWS region.

### Step 5: Create VPC

A VPC was defined with a CIDR block to establish the network environment.

Example:

CIDR Block: 10.0.0.0/16

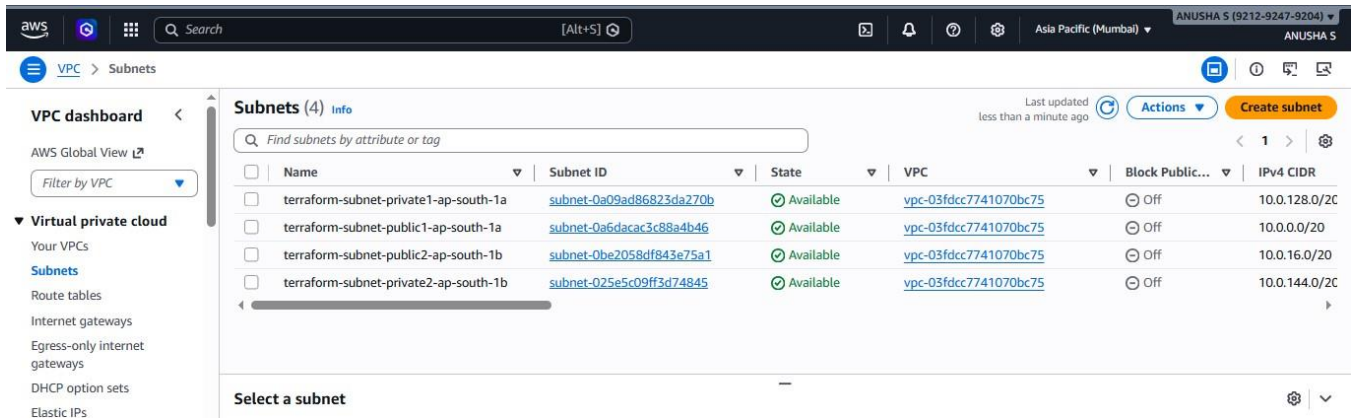


## Step 6: Create Subnets

Two public subnets were created in different availability zones.

Purpose:

To allow the load balancer to distribute traffic across zones.



**Subnets (4)** info

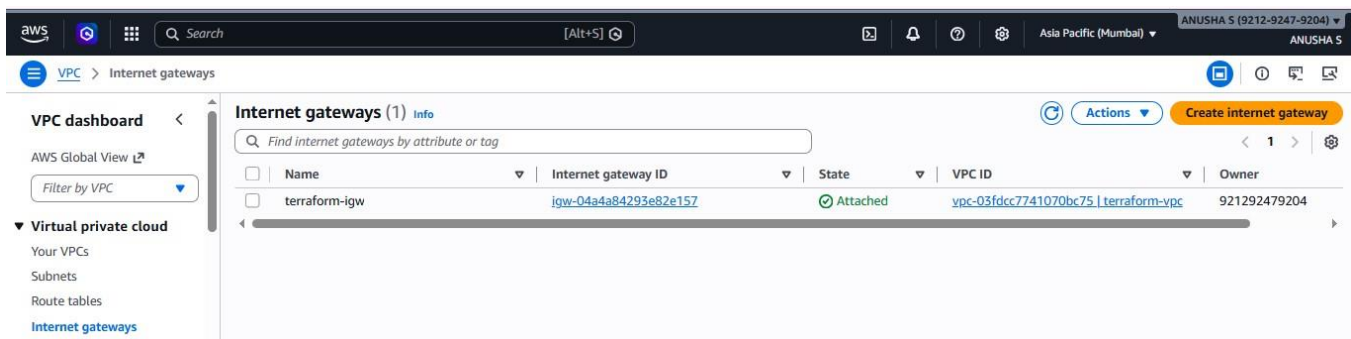
Find subnets by attribute or tag

| Name                                  | Subnet ID                                | State     | VPC                                   | Block Public... | IPv4 CIDR     |
|---------------------------------------|------------------------------------------|-----------|---------------------------------------|-----------------|---------------|
| terraform-subnet-private1-ap-south-1a | <a href="#">subnet-0a09ad86823da270b</a> | Available | <a href="#">vpc-03fdcc7741070bc75</a> | Off             | 10.0.128.0/20 |
| terraform-subnet-public1-ap-south-1a  | <a href="#">subnet-0a6dacac3c88a4b46</a> | Available | <a href="#">vpc-03fdcc7741070bc75</a> | Off             | 10.0.0.0/20   |
| terraform-subnet-public2-ap-south-1b  | <a href="#">subnet-0be2058df843e75a1</a> | Available | <a href="#">vpc-03fdcc7741070bc75</a> | Off             | 10.0.16.0/20  |
| terraform-subnet-private2-ap-south-1b | <a href="#">subnet-025e5c09ff3d74845</a> | Available | <a href="#">vpc-03fdcc7741070bc75</a> | Off             | 10.0.144.0/20 |

Select a subnet

## Step 7: Create Internet Gateway

An Internet Gateway was attached to the VPC to enable internet access.



**Internet gateways (1)** info

Find internet gateways by attribute or tag

| Name          | Internet gateway ID                   | State    | VPC ID                                                                | Owner        |
|---------------|---------------------------------------|----------|-----------------------------------------------------------------------|--------------|
| terraform-igw | <a href="#">igw-04a4a84293e82e157</a> | Attached | <a href="#">vpc-03fdcc7741070bc75</a>   <a href="#">terraform-vpc</a> | 921292479204 |

## Step 8: Configure Route Tables

Route tables were created and associated with the subnets to direct traffic to the Internet Gateway.

**Route tables (4)** Info

Find route tables by attribute or tag

| Name                               | Route table ID        | Explicit subnet associ... | Edge associations | Main | VPC                         |
|------------------------------------|-----------------------|---------------------------|-------------------|------|-----------------------------|
| terraform-rtb-private2-ap-south-1b | rtb-09a459e111cd8213c | subnet-025e5c09ff3d748... | -                 | No   | vpc-03fdcc7741070bc75   ter |
| -                                  | rtb-00778cd8936f7531c | -                         | -                 | Yes  | vpc-03fdcc7741070bc75   ter |
| terraform-rtb-private1-ap-south-1a | rtb-088c81946ce0961ff | subnet-0a09ad86823da2...  | -                 | No   | vpc-03fdcc7741070bc75   ter |
| terraform-rtb-public               | rtb-08b9937f59faaab04 | 2 subnets                 | -                 | No   | vpc-03fdcc7741070bc75   ter |

Select a route table

## Step 9: Create Security Groups

Security groups were configured to allow HTTP and SSH access.

**Security Groups (57)** Info

Find security groups by attribute or tag

| Name | Security group ID    | Security group name | VPC ID             |
|------|----------------------|---------------------|--------------------|
| -    | sg-0b7aa84b39981ba8d | launch-wizard-42    | vpc-0e1534f6e8f... |
| -    | sg-04ef0fbcd29c94d9b | launch-wizard-18    | vpc-0e1534f6e8f... |
| -    | sg-0e67f9daf6b520000 | launch-wizard-16    | vpc-0e1534f6e8f... |

## Step 10: Launch EC2 Instance

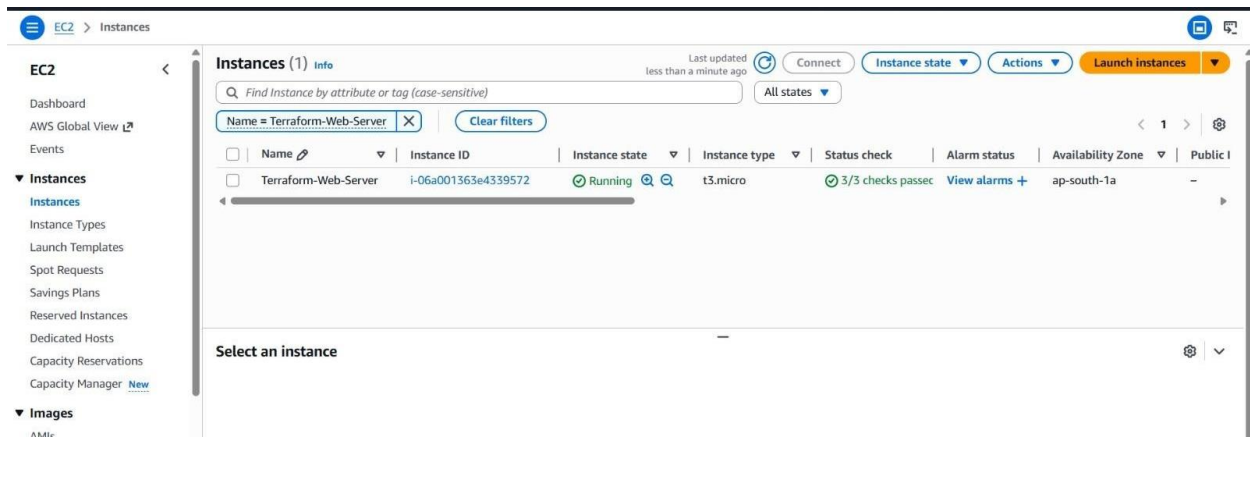
An EC2 instance was launched within the public subnet to host the Apache web server.

Apache was installed using:

```
sudo apt update
sudo apt install apache2 -y
```

A test webpage was created.



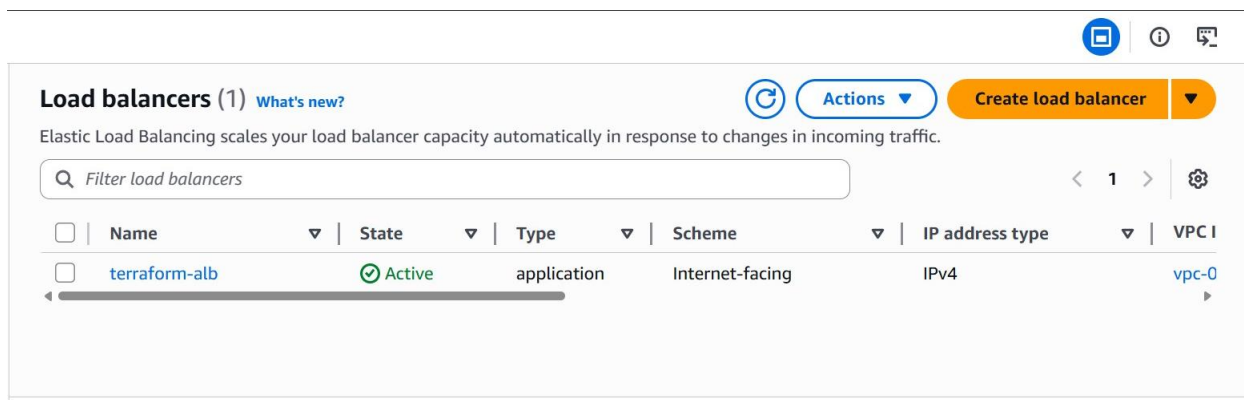


## Step 11: Create Application Load Balancer

An ALB was configured across two public subnets.

Purpose:

To distribute traffic and improve availability.



## Step 12: Create Target Group

A target group was created and the EC2 instance was registered.

Health checks were configured to verify instance availability.

**Target groups (1)**
[Info](#)
[What's new?](#)

Actions ▾

Create target group

< 1 >

| <input type="checkbox"/> | Name ▾                                 | ARN ▾                          | Port ▾ | Protocol ▾ | Target type ▾ |
|--------------------------|----------------------------------------|--------------------------------|--------|------------|---------------|
| <input type="checkbox"/> | <a href="#">terraform-target-group</a> | arn:aws:elasticloadbalancin... | 80     | HTTP       | Instance      |

**Targets**
Monitoring
Health checks
Attributes
Tags

**Registered targets (1)**
[Info](#)

Anomaly mitigation: **Not applicable**

Deregister
Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

< 1 >

| <input type="checkbox"/> | Instance ID ▾                       | Name ▾          | Port ▾ | Zone ▾            | Health status ▾ | Health sta |
|--------------------------|-------------------------------------|-----------------|--------|-------------------|-----------------|------------|
| <input type="checkbox"/> | <a href="#">i-0112b68f4702cbcec</a> | Terraform-We... | 80     | ap-south-1a (a... | Healthy         | -          |

## Step 13: Configure Listener

An HTTP listener on port 80 was created to forward requests to the target group.

**Listeners and rules (1)**
[Info](#)

Manage rules ▾
Manage listener ▾

Add listener

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

< 1 >

| <input type="checkbox"/> | Protocol:Port ▾         | Default action ▾                                                                                                                                                                                                     | Rules ▾                | ARN ▾ | Security policy ▾ |
|--------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|-------|-------------------|
| <input type="checkbox"/> | <a href="#">HTTP:80</a> | <ul style="list-style-type: none"> <li>Forward to target group <ul style="list-style-type: none"> <li><a href="#">terraform-target-group</a> : 1 (100%)</li> <li>Target group stickiness: Off</li> </ul> </li> </ul> | <a href="#">1 rule</a> | ARN   | Not applicable    |

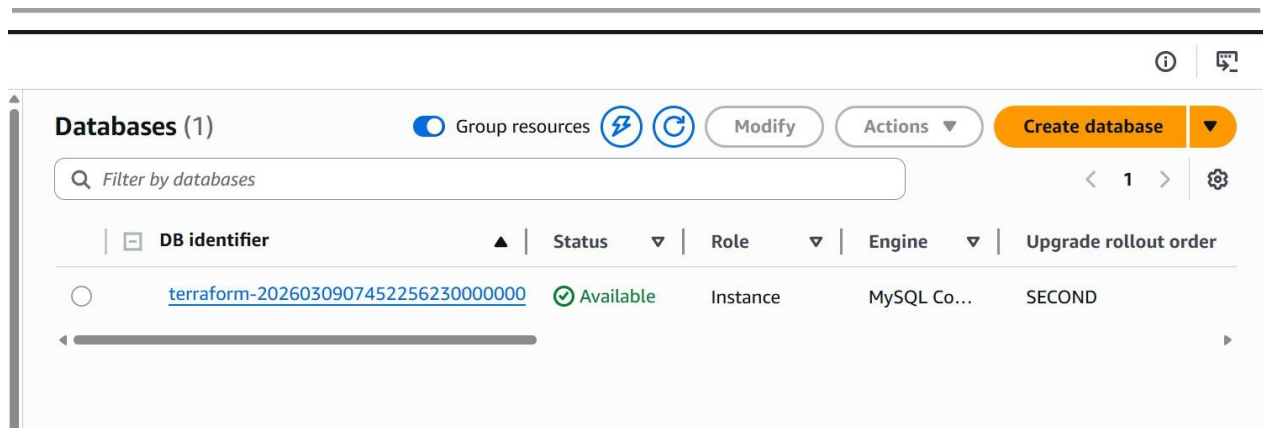
## Step 14: Create RDS Database

An RDS database instance was created to store application data.

Configuration included:

- Database engine: MySQL
- Instance type: db.t3.micro
- Storage: 20GB

The database was deployed inside the VPC and secured using security groups.



## Step 14: Deploy Infrastructure

Terraform commands were executed:

```
terraform init
terraform plan
terraform apply
```

This created all infrastructure resources automatically.

```
ubuntu@ip-172-31-6-252:~/terraform-aws-project$ terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
ubuntu@ip-172-31-6-252:~/terraform-aws-project$ terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# aws_db_instance.mydb will be created
+ resource "aws_db_instance" "mydb" {
  + address                        = (known after apply)
  + allocated_storage             = 20
  + apply_immediately             = false
  + arn                           = (known after apply)
  + auto_minor_version_upgrade   = true
  + availability_zone             = (known after apply)
  + backup_retention_period       = (known after apply)
  + backup_target                 = (known after apply)
  + backup_window                 = (known after apply)
  + ca_cert_identifier            = (known after apply)
  + character_set_name            = (known after apply)
  + copy_tags_to_snapshot        = false
  + database_insights_mode       = (known after apply)
  + db_name                       = (known after apply)
```

```
  + enable_dns_support           = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                           = (known after apply)
  + instance_tenancy             = "default"
  + ipv6_association_id          = (known after apply)
  + ipv6_cidr_block              = (known after apply)
  + ipv6_cidr_block_network_border_group = (known after apply)
  + main_route_table_id          = (known after apply)
  + owner_id                     = (known after apply)
  + region                       = "ap-south-1"
  + tags                         = {
    + "Name" = "terraform-vpc"
  }
  + tags_all                     = {
    + "Name" = "terraform-vpc"
  }
}
```

**Plan:** 10 to add, 0 to change, 0 to destroy.

---

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run `"terraform apply"` now.

```

aws_db_instance.mydb: Still creating... [01m40s elapsed]
aws_db_instance.mydb: Still creating... [01m50s elapsed]
aws_db_instance.mydb: Still creating... [02m00s elapsed]
aws_db_instance.mydb: Still creating... [02m10s elapsed]
aws_db_instance.mydb: Still creating... [02m20s elapsed]
aws_db_instance.mydb: Still creating... [02m30s elapsed]
aws_db_instance.mydb: Still creating... [02m40s elapsed]
aws_db_instance.mydb: Still creating... [02m50s elapsed]
aws_db_instance.mydb: Still creating... [03m00s elapsed]
aws_db_instance.mydb: Still creating... [03m10s elapsed]
aws_db_instance.mydb: Still creating... [03m20s elapsed]
aws_db_instance.mydb: Still creating... [03m30s elapsed]
aws_db_instance.mydb: Still creating... [03m40s elapsed]
aws_db_instance.mydb: Still creating... [03m50s elapsed]
aws_db_instance.mydb: Still creating... [04m00s elapsed]
aws_db_instance.mydb: Still creating... [04m10s elapsed]
aws_db_instance.mydb: Still creating... [04m20s elapsed]
aws_db_instance.mydb: Still creating... [04m30s elapsed]
aws_db_instance.mydb: Still creating... [04m40s elapsed]
aws_db_instance.mydb: Still creating... [04m50s elapsed]
aws_db_instance.mydb: Still creating... [05m00s elapsed]
aws_db_instance.mydb: Creation complete after 5m4s [id=db-3GRXARYL4CA7OJYVD2CN5X2F3A]

Apply complete! Resources: 10 added, 0 changed, 0 destroyed.

```

## Step 16: Verify Application

The DNS of the Application Load Balancer was opened in a browser.

Example:

<http://terraform-alb-xxxx.ap-south-1.elb.amazonaws.com>



## EC2-Instance-Public Ip



The webpage displayed:

Terraform AWS Project Working

This confirms the infrastructure deployment was successful.

## 7. Troubleshooting

| Issue                   | Method                              | Solution                            |
|-------------------------|-------------------------------------|-------------------------------------|
| EC2 website not loading | Tested using curl                   | Installed Apache                    |
| Target group unhealthy  | Checked health check settings       | Started web server                  |
| 502 Bad Gateway         | Verified security groups and routes | Allowed ALB traffic                 |
| ALB health check failed | Tested instance IP connectivity     | Installed web server on correct EC2 |
| Connection refused      | Checked server status               | Restarted Apache                    |

## 8. Conclusion

This project successfully demonstrated how cloud infrastructure can be automated using Terraform on AWS. The infrastructure components including VPC, subnets, security groups, EC2 instance, and Application Load Balancer were deployed using Infrastructure as Code principles.

By automating infrastructure provisioning, the deployment process becomes faster, more reliable, and easier to maintain. This approach is widely used in modern DevOps practices to ensure scalability, consistency, and efficient infrastructure management.

The project highlights the importance of automation in cloud environments and provides practical experience in managing AWS infrastructure using Terraform.